

Lab2：内存管理

github仓库：[实验二github网址](#)

一、实验目的

- 理解 **RISC-V 虚拟内存管理机制**（SV39 分级页表结构）。
- 实现 **物理内存分配器**，支持内核区和用户区的分离管理。
- 构建 **内核页表**，并完成内核地址空间到物理地址的映射。
- 熟悉 **虚拟内存相关函数实现流程**：`pmem_alloc`、`peme_free`、`vm_getpte`、`vm_mappages`、`vm_unmappages`。
- 通过测试验证 **内存管理与页表启用**是否正确。

二、系统设计部分

1. 架构设计说明

- **物理内存管理**
 - 以**空闲页链表**维护可用页；每个物理页的起始 8 字节存放指向下一空闲页的指针。
 - 将物理内存划分为**内核页区**与**用户页区**，以避免用户进程耗尽物理页、危及内核运行安全。
- **虚拟内存管理**
 - 采用 **SV39**：39 位虚拟地址，**三级页表**，每级索引占 9 位。
 - 内核页表（`kernel_pgtbl`）负责映射：
 1. **设备寄存器区域**：`0x100000000 ~ 0x200000000`
 2. **内存区域**：`0x800000000 ~ 0x800000000 + 128MB`
 - 页表使用**一一对应映射（VA=PA）**以简化实现。

2. 关键数据结构

(1) 物理页管理

```
// 空闲页节点
typedef struct page_node {
    struct page_node* next;
} page_node_t;
```

```
// 可分配区域
```

```
typedef struct alloc_region {
    uint64 begin; // 起始的物理地址
    uint64 end; // 结束的物理地址
    spinlock_t lk; // 自旋锁
    uint32 allocable; // 可以分配的自旋锁页数
    page_node_t list_head; // 头节点
} alloc_region_t;
```

(2) 虚拟内存管理

```
typedef uint64 pte_t; // 页表项（含物理页号和标志位）
typedef uint64* pgtbl_t; // 顶级页表指针
```

页表项存放物理页号及标志位（R/W/X/V），页表本身也是物理页。

3. 与 xv6 对比分析

方面	本实验实现	xv6 实现
物理内存管理	使用链表头插法，分离内核和用户区域	使用空闲链表，但不区分内核/用户
页表结构	RISC-V SV39 三级页表	x86 单级页表（或简化多级）
内核页表	VA=PA 直接映射硬件区和内存区	同样建立内核恒等映射
设计目标	防止用户进程耗尽物理页、支持并发安全	重点在简单支持多进程内存管理

4. 设计决策理由

- 内核/用户页区分：避免恶意用户耗尽内存。
- 链表头插法：实现简单，`alloc` / `free` 均为 $O(1)$ 。
- 直接映射：减少调试难度，保证内核访问稳定。
- 三级页表：符合 RISC-V SV39 规范，节省空间，利于扩展。

三、实验过程

1. 实现步骤记录

- 物理内存管理
 - 定义 `page_node_t` 与 `alloc_region_t`。
 - 完成 `pmem_init`：分别构建内核区与用户区的空闲页链表。
 - 实现 `pmem_alloc` / `pmem_free`：支持页的分配与回收。

- **虚拟内存管理**
 - 实现 `vm_getpte`：按级查找/按需分配页表项。
 - 实现 `vm_mappages`：建立 VA→PA 的映射关系。
 - 实现 `vm_unmappages`：解除映射，且可选释放对应物理页。
 - 实现 `vm_print`：输出页表与映射信息，便于调试。
- **内核页表构建**
 - 在 `kvm_init` 中申请并初始化 `kernel_pgtbl`。
 - 映射设备寄存器区与物理内存区。
 - 通过 `kvm_inithart` 写入 `satp`，随后执行 `sfence_vma` 以启用页表生效。

2. 问题与解决方案

- **问题一**：在处理物理内存时发现，原先的库函数panic有误，无法反映出错误的信息。
解决方案：发现多赋值了一次panicked，修改后问题消失。
- **问题二**：`vm_mappages` 函数重复映射问题
解决方案：暂时允许覆盖层
- **问题三**：`vm_unmappages` 出现释放物理页与虚拟页不对应的情况
解决方案：每次释放前要判断是内核页还是用户页

3. 源码理解小结

- **pmem.c**：负责物理页管理，使用**链表头插**维护空闲页。
- **vmem.c**：实现虚拟内存的核心流程，涵盖页表查找、创建映射与解除映射。

总体思路：内核先用 `pmem_alloc` 申请物理页用作各级页表；再由 `vm_mappages` 建立 VA→PA 关联；最终写入 `satp` 启用 MMU，让地址转换生效。

四、测试验证部分

物理页测试，在 main 中输入测试代码，可以看到成功是实现 alloc 和 free

```
mem = 0x00000000826b5000, data = 16843009
mem = 0x00000000826b4000, data = 16843009
mem = 0x00000000826b3000, data = 16843009
mem = 0x00000000826b2000, data = 16843009
cpu 0 alloc over
mem = 0x00000000826b1000, data = 16843009
mem = 0x00000000826b0000, data = 16843009
mem = 0x00000000826af000, data = 16843009
mem = 0x00000000826ae000, data = 16843009
cpu 1 alloc over
cpu 1 free over
cpu 0 free over
```

虚拟内存测试

```
VA 0x0000000000000000 -> PA 0x0000000087fff000 flags 0x3
VA 0x0000000000000a000 -> PA 0x0000000087ffe000 flags 0x7
VA 0x00000000000200000 -> PA 0x0000000087ffd000 flags 0xb
VA 0x00000000040000000 -> PA 0x0000000087ffd000 flags 0xb
VA 0x00000003fffffff000 -> PA 0x0000000087ffb000 flags 0x7
```

test-2

```
VA 0x0000000000000000 -> PA 0x0000000087fff000 flags 0x7
VA 0x00000000400000000 -> PA 0x0000000087ffd000 flags 0xb
VA 0x00000003fffffff000 -> PA 0x0000000087ffb000 flags 0x7
```

□

页表启用后，系统仍能输出调试信息，说明地址翻译正确。